part F. Arrays

JavaScript arrays are wildly flexible and can hold any mixture of values inside:

```
ContentViewer, zrr, ChapterReader x2
const elements = [true, null, undefined, 42];

elements.push("even", ["more"]);
// Value of elements: [true, null, undefined, 42, "even", ["more"]]
```

In most cases, though, individual JavaScript arrays are intended to hold only one specific type of value. Adding values of a different type may be confusing to readers, or worse, the result of an error that could cause problems in the program.

TypeScript respects the best practice of keeping to one data type per array by remembering what type of data is initially inside an array, and only allowing the array to operate on that kind of data.

In this example, TypeScript knows the `warriors` array initially contains `string` typed values, so while adding more `string` typed values is allowed, adding any other type of data is not:

```
const warriors = ["Artemisia", "Boudica"];

// Ok: "Zenobia" is a string
ser x2
warriors.push("Zenobia");

warriors.push(true);
//            ~~~~
// Argument of type 'boolean' is not assignable to parameter of type 'string'.
```

You can think of TypeScript's inference of an array's type from its initial members as similar to how it understands variable types from their initial values. TypeScript generally tries to understand the intended types of your code from how values are assigned, and arrays are no exception.

Array Types

As with other variable declarations, variables meant to store arrays don't need to have an initial value. The variables can start off `undefined` and receive an array value later.

TypeScript will want you to let it know what types of values are meant to go in the array by giving the variable a type annotation. The type annotation for an array requires the type of elements in the array followed by a `[]`:

```
let arrayOfNumbers: number[];

arrayOfNumbers = [4, 8, 15, 16, 23, 42];
```

Array and Function Types

Array types are an example of a syntax container where function types may need parentheses to distinguish what's in the function type or not. Parentheses may be used to indicate which part of an annotation is the function return or the surrounding array type.

The `createStrings` type here, which is a function type, is not the same as `stringCreators`, which is an array type:

```
// Type is a function that returns an array of strings
let createStrings: () => string[];

// Type is an array of functions that each return a string
let stringCreators: (() => string)[];
```

Union-Type Arrays

You can use a union type to indicate that each element of an array can be one of multiple select types.

When using array types with unions, parentheses may need to be used to indicate which part of an annotation is the contents of the array or the surrounding union type. Using parentheses in array union types is important—the following two types are not the same:

```ts
// Type is either a string or an array of numbers
let stringOrArrayOfNumbers: string | number[];

// Type is an array of elements that are each either a number or a string
let arrayOfStringOrNumbers: (string | number)[];
```

TypeScript will understand from an array's declaration that it is a union-type array if it contains more than one type of element. In other words, the type of an array's elements is the union of all possible types for elements in the array.

Here, `namesMaybe` is `(string | undefined)[]` because it has both `string` values and an `undefined` value:

```ts
// Type is (string | undefined)[]
const namesMaybe = [
  "Aqualtune",
  "Blenda",
  undefined,
];
```

Evolving Any Arrays

If you don't include a type annotation on a variable initially set to an empty array, TypeScript will treat the array as evolving `any[]`, meaning it can receive any content. As with evolving `any` variables, we don't like evolving `any[]` arrays. They partially negate the benefits of TypeScript's type checker by allowing you to add potentially incorrect values.

This `values` array starts off containing `any` elements, evolves to contain `string` elements, then again evolves to include `number | string` elements:

```ts
// Type: any[]
let values = [];

// Type: string[]
values.push('');

// Type: (number | string)[]
values[0] = 0;
```

As with variables, allowing arrays to be evolving `any` typed —and using the `any` type in general—partially defeats the

purpose of TypeScript's type checking. TypeScript works best when it knows what types your values are meant to be.

Multidimensional Arrays

A 2D array, or an array of arrays, will have two "[]"s:

```
let arrayOfArraysOfNumbers: number[][];

arrayOfArraysOfNumbers = [
  [1, 2, 3],
  [2, 4, 6],
  [3, 6, 9],
];
```

A 3D array, or an array of arrays of arrays, will have three "[]"s. 4D arrays have four "[]"s. 5D arrays have five "[]"s. You can guess where this is going for 6D arrays and beyond.

These multidimensional array types don't introduce any new concepts to array types. Think of a 2D array as taking in the original type, which just so happens to have `[]` at the end, and adding a `[]` after it.

This `arrayOfArraysOfNumbers` array is of type `number[][]`, which is also representable by `(number[])[]`:

```
// Type: number[][]
let arrayOfArraysOfNumbers: (number[])[];
```

Array Members

TypeScript understands typical index-based access for retrieving members of an array to give back an element of that array's type.

This `defenders` array is of type `string[]`, so `defender` is a `string`:

```
const defenders = ["Clarenza", "Dina"];

// Type: string
const defender = defenders[0];
```

Members of union typed arrays are themselves that same union type.

Here, `soldiersOrDates` is of type `(string | Date)[]`, so the `soldierOrDate` variable is of type `string | Date`:

```
const soldiersOrDates = ["Deborah Sampson", new Date(1782, 6, 3)];

// Type: Date | string
const soldierOrDate = soldiersOrDates[0];
```

Caveat: Unsound Members

The TypeScript type system is known to be technically *unsound*: it can get types mostly right, but sometimes its understanding about the types of values may be incorrect. Arrays in particular are a source of unsoundness in the type system. By default, TypeScript assumes all array member accesses return a member of that array, even though in JavaScript, accessing an array element with an index greater than the array's length gives `undefined`.

This code gives no complaints with the default TypeScript compiler settings:

```
function withElements(elements: string[]) {
  console.log(elements[9001].length); // No type error
}

withElements(["It's", "over"]);
```

We as readers can deduce that it'll crash at runtime with "`Cannot read property 'length' of undefined`", but TypeScript intentionally does not make sure retrieved array members exist. It sees `elements[9001]` in the code snippet as being type `string`, not `undefined`.

Spreads and Rests

Remember `...` rest parameters for functions from ? Rest parameters and array spreading, both with the `...` operator, are key ways to interact with arrays in JavaScript. TypeScript understands both of them.

Spreads

Arrays can be joined together using the `...` spread operator. TypeScript understands the result array will contain values that can be from either of the input arrays.

If the input arrays are the same type, the output array will be that same type. If two arrays of different types are spread together to create a new array, the new array will be understood to be a union type array of elements that are either of the two original types.

Here, the `conjoined` array is known to contain both values that are type `string` and values that are type `number`, so its type is inferred to be `(string | number)[]`:

```
// Type: string[]
const soldiers = ["Harriet Tubman", "Joan of Arc", "Khutulun"];

// Type: number[]
const soldierAges = [90, 19, 45];

// Type: (string | number)[]
const conjoined = [...soldiers, ...soldierAges];
```

Spreading Rest Parameters

TypeScript recognizes and will perform type checking on the JavaScript practice of `...` spreading an array as a rest parameter. Arrays used as arguments for rest parameters must have the same array type as the rest parameter.

The `logWarriors` function below takes in only `string` values for its `...names` rest parameter. Spreading an array of type `string[]` is allowed, but a `number[]` is not:

```
function logWarriors(greeting: string, ...names: string[]) {
  for (const name of names) {
    console.log(`${greeting}, ${name}!`);
  }
}

const warriors = ["Cathay Williams", "Lozen", "Nzinga"];

logWarriors("Hello", ...warriors);
```

```
const birthYears = [1844, 1840, 1583];

logWarriors("Born in", ...birthYears);
//                      ~~~~~~~~~~~~~
// Error: Argument of type 'number' is not
// assignable to parameter of type 'string'.
```

Tuples

Although JavaScript arrays may be any size in theory, it is sometimes useful to use an array of a fixed size—also known as a *tuple*. Tuple arrays have a specific known type at each index that may be more specific than a union type of all possible members of the array. The syntax to declare a tuple type looks like an array literal, but with types in place of element values.

Here, the array `yearAndWarrior` is declared as being a tuple type with a `number` at index 0 and a `string` at index 1:

```
let yearAndWarrior: [number, string];

yearAndWarrior = [530, "Tomyris"]; // Ok

yearAndWarrior = [false, "Tomyris"];
//                ~~~~~
// Error: Type 'boolean' is not assignable to type 'number'.

yearAndWarrior = [530];
// Error: Type '[number]' is not assignable to type '[number, string]'.
//   Source has 1 element(s) but target requires 2.
```

Tuples are often used in JavaScript alongside array destructuring to be able to assign multiple values at once, such as setting two variables to initial values based on a single condition.

For example, TypeScript recognizes here that `year` is always going to be a `number` and `warrior` is always going to be a `string`:

```
// year type: number
// warrior type: string
let [year, warrior] = Math.random() > 0.5
```

```
    ? [340, "Archidamia"]
    : [1828, "Rani of Jhansi"];
```

Tuple Assignability

Tuple types are treated by TypeScript as more specific than variable length array types. That means variable length array types aren't assignable to tuple types.

Here, although we as humans may see `pairLoose` as having `[boolean, number]` inside, TypeScript infers it to be the more general `(boolean | number)[]` type:

```
// Type: (boolean | number)[]
const pairLoose = [false, 123];

const pairTupleLoose: [boolean, number] = pairLoose;
//    ~~~~~~~~~~~~~~
// Error: Type '(number | boolean)[]' is not
// assignable to type '[boolean, number]'.
//   Target requires 2 element(s) but source may have fewer.
```

If `pairLoose` had been declared as a `[boolean, number]` itself, the assignment of its value to `pairTuple` would have been permitted.

Tuples of different lengths are also not assignable to each other, as TypeScript includes knowing how many members are in the tuple in tuple types.

Here, `tupleTwoExtra` must have exactly two members, so although `tupleThree` starts with the correct members, its third member prevents it from being assignable to `tupleTwoExtra`:

```
const tupleThree: [boolean, number, string] = [false, 1583, "Nzinga"];

const tupleTwoExact: [boolean, number] = [tupleThree[0], tupleThree[1]];

const tupleTwoExtra: [boolean, number] = tupleThree;
//    ~~~~~~~~~~~~~
// Error: Type '[boolean, number, string]' is
// not assignable to type '[boolean, number]'.
//   Source has 3 element(s) but target allows only 2.
```

Tuples as rest parameters

Because tuples are seen as arrays with more specific type information on length and element types, they can be particularly useful for storing arguments to be passed to a function. TypeScript is able to provide accurate type checking for tuples passed as `...` rest parameters.

Here, the `logPair` function's parameters are typed `string` and `number`. Trying to pass in a value of type `(string | number)[]` as arguments wouldn't be type safe as the contents might not match up: they could both be the same type, or one of each type in the wrong order. However, if TypeScript knows the value to be a `[string, number]` tuple, it understands the values match up:

```
function logPair(name: string, value: number) {
  console.log(`${name} has ${value}`);
}

const pairArray = ["Amage", 1];

logPair(...pairArray);
// Error: A spread argument must either have a
// tuple type or be passed to a rest parameter.

const pairTupleIncorrect: [number, string] = [1, "Amage"];

logPair(...pairTupleIncorrect);
// Error: Argument of type 'number' is not
// assignable to parameter of type 'string'.

const pairTupleCorrect: [string, number] = ["Amage", 1];

logPair(...pairTupleCorrect); // Ok
```

If you really want to go wild with your rest parameters tuples, you can mix them with arrays to store a list of arguments for multiple function calls. Here, `trios` is an array of tuples, where each tuple also has a tuple for its second member. `trios.forEach(trio => logTrio(...trio))` is known to be safe because each `...trio` happens to match the parameter types of `logTrio`. `trios.forEach(logTrio)`, however, is not assignable because that is attempting to pass the entire `[string, [number, boolean]]` as the first parameter, which is type `string`:

```ts
function logTrio(name: string, value: [number, boolean]) {
  console.log(`${name} has ${value[0]} (${value[1]})`);
}

const trios: [string, [number, boolean]][] = [
  ["Amanitore", [1, true]],
  ["Æthelflæd", [2, false]],
  ["Ann E. Dunwoody", [3, false]]
];

trios.forEach(trio => logTrio(...trio)); // Ok

trios.forEach(logTrio);
//            ~~~~~~~
// Argument of type '(name: string, value: [number, boolean]) => void'
// is not assignable to parameter of type
// '(value: [string, [number, boolean]], ...) => void'.
//   Types of parameters 'name' and 'value' are incompatible.
//     Type '[string, [number, boolean]]' is not assignable to type 'string'.
```

Tuple Inferences

TypeScript generally treats created arrays as variable length arrays, not tuples. If it sees an array being used as a variable's initial value or the returned value for a function, then it will assume a flexible size array rather than a fixed size tuple.

The following `firstCharAndSize` function is inferred as returning `(string | number)[]`, not `[string, number]`, because that's the type inferred for its returned array literal:

```ts
// Return type: (string | number)[]
function firstCharAndSize(input: string) {
  return [input[0], input.length];
}

// firstChar type: string | number
// size type: string | number
const [firstChar, size] = firstCharAndSize("Gudit");
```

There are two common ways in TypeScript to indicate that a value should be a more specific tuple type instead of a general array type: explicit tuple types and `const` assertions.

Explicit tuple types

Tuple types may be used in type annotations, such as the return type annotation for a function. If the function is declared as returning a tuple type and returns an array literal, that array literal will be inferred to be a tuple instead of a more general variable-length array.

This `firstCharAndSizeExplicit` function version explicitly states that it returns a tuple of a `string` and `number`:

```
// Return type: [string, number]
function firstCharAndSizeExplicit(input: string): [string, number] {
  return [input[0], input.length];
}

// firstChar type: string
// size type: number
const [firstChar, size] = firstCharAndSizeExplicit("Cathay Williams");
```

Const asserted tuples

Typing out tuple types in explicit type annotations can be a pain for the same reasons as typing out any explicit type annotations. It's extra syntax for you to write and update as code changes.

As an alternative, TypeScript provides an `as const` operator known as a *const assertion* that can be placed after a value. Const assertions tell TypeScript to use the most literal, read-only possible form of the value when inferring its type. If one is placed after an array literal, it will indicate that the array should be treated as a tuple:

```
// Type: (string | number)[]
const unionArray = [1157, "Tomoe"];

// Type: readonly [1157, "Tomoe"]
const readonlyTuple = [1157, "Tomoe"] as const;
```

Note that `as const` assertions go beyond switching from flexible sized arrays to fixed size tuples: they also indicate to TypeScript that the tuple is read-only and cannot be used in a place that expects it should be allowed to modify the value.

In this example, `pairMutable` is allowed to be modified because it has a traditional explicit tuple type. However, the `as const` makes the value not assignable to the mutable `pairAlsoMutable`, and members of the constant `pairConst` are not allowed to be modified:

```
const pairMutable: [number, string] = [1157, "Tomoe"];
pairMutable[0] = 1247; // Ok

const pairAlsoMutable: [number, string] = [1157, "Tomoe"] as const;
//    ~~~~~~~~~~~~~~~
// Error: The type 'readonly [1157, "Tomoe"]' is 'readonly'
// and cannot be assigned to the mutable type '[number, string]'.

const pairConst = [1157, "Tomoe"] as const;
pairConst[0] = 1247;
//        ~
// Error: Cannot assign to '0' because it is a read-only property.
```

In practice, read-only tuples are convenient for function returns. Returned values from functions that return a tuple are often destructured immediately anyway, so the tuple being read-only does not get in the way of using the function.

This `firstCharAndSizeAsConst` returns a `readonly [string, number]`, but the consuming code only cares about retrieving the values from that tuple:

```
// Return type: readonly [string, number]
function firstCharAndSizeAsConst(input: string) {
  return [input[0], input.length] as const;
}

// firstChar type: string
// size type: number
const [firstChar, size] = firstCharAndSizeAsConst("Ching Shih");
```

In this part, you worked with declaring arrays and retrieving their members:

- Declaring array types with `[]`
- Using parentheses to declare arrays of functions or union types
- How TypeScript understands array elements as the type of the array
- Working with `...` spreads and rests

- Declaring tuple types to represent fixed-size arrays
- Using type annotations or `as const` assertions to create tuples